Edge — a Django view — Solution

> Back to the exercise · Hint 1 · Hint 2

Story: this builds the actual shape of a Django project — a view function, a `urls.py` entry routing to it, and the project-level settings tying it together — so you genuinely feel why Django carries more structure than Flask, and recognize when its built-in batteries (auth, the admin panel, the ORM) are actually worth that extra weight. **If not:** you'd reach for Django on a weekend demo where it's pure overkill, or avoid it on a real product where it would have saved you weeks of work.

The shape of a real Django app

In a real project you run `django-admin startproject site` and get this layout — you don't write it by hand:

```
site/
├── manage.py
├── site/           settings.py, urls.py (project config)
└── classifier/     views.py, urls.py, models.py (your app)
```

Your two files are tiny:

```
# classifier/views.py
import json

from django.http import JsonResponse

from pipeline import classify

def classify_view(request):
    try:
        body = json.loads(request.body)
    except json.JSONDecodeError:
        return JsonResponse({"error": "invalid JSON"}, status=400)
    text = body.get("text", "")
    if not text.strip():
        return JsonResponse({"error": "empty text"}, status=400)
    return JsonResponse({"label": classify(text)})
```

```
# classifier/urls.py
from django.urls import path

from .views import classify_view

urlpatterns = [path("classify", classify_view)]
```

Then you'd run `python manage.py runserver`. The view and url are all your AI code; everything else (the server, routing machinery, settings) Django provides.

A single-file runnable version (for testing)

To run and test the view without a full project, configure Django inline — this is the same `classify_view`, just bootstrapped in one file:

```
# practice/django_app.py   —   python practice/django_app.py
import json

from django.conf import settings

settings.configure(
    DEBUG=True, ROOT_URLCONF=__name__, ALLOWED_HOSTS=["*"],
    DATABASES={}, INSTALLED_APPS=[],
)

import django
django.setup()

from django.http import JsonResponse
from django.urls import path

from pipeline import classify

def classify_view(request):
    try:
        body = json.loads(request.body)
    except json.JSONDecodeError:
        return JsonResponse({"error": "invalid JSON"}, status=400)
    text = body.get("text", "")
    if not text.strip():
        return JsonResponse({"error": "empty text"}, status=400)
    return JsonResponse({"label": classify(text)})

urlpatterns = [path("classify", classify_view)]
```

```
def main():
    from django.test import Client
    client = Client()
    good = client.post("/classify", data=json.dumps({"text": "amazing"}),
                       content_type="application/json")
    empty = client.post("/classify", data=json.dumps({"text": ""}),
                        content_type="application/json")
    ok = good.json() == {"label": "positive"} and empty.status_code == 400
    print(f"case 1 (view + validation) -> {'OK' if ok else 'WRONG'}")

if __name__ == "__main__":
    main()
```

Expected: `POST /classify {"text": "amazing"}` returns `{"label": "positive"}`; empty text returns `400`.

Why Django is heavier — and when that's right

- The single endpoint above took more ceremony than Flask or FastAPI — that's Django being honest about what it is. You don't choose it for *one* endpoint.
- You choose it when the app is a **product**: `python manage.py createsuperuser` and `/admin` give you a full CRUD interface over your data (saved conversations, users, feedback) with no code; the ORM gives you database models as Python classes with migrations; the auth system gives you logins for free.
- Rule of thumb: a demo is Streamlit, a small app is Flask, an API is FastAPI, and a **product with users, stored data, and an admin** is Django. The AI pipeline inside is identical in all four.